Universidade Federal de Viçosa – Campus Florestal
Projeto Integrador 2025 - Arquitetura de Software
Equipe 2

Documento de Arquitetura de Software

Este documento pretende fornecer uma versão inicial da arquitetura do sistema do jogo “Pingo: Uma Aventura da Rede”, idealizado e desenvolvido pela Equipe 2 durante o Projeto Integrador 2025.

Versão: 1.0

Data: 01/09/2025

Autor(a): Ana Carolina Fernandes, Arquiteta de Software

1. Visão Geral da Arquitetura.....2
1.1. Padrão Arquitetural Adotado.....2
1.2. Organização do Projeto..... 3
2. Definição das Camadas do Backend..... 4
2.1. Controller (Camada de Apresentação)..... 4
2.2. Service (Camada de Negócio).....5
2.3. Data Access Object (Camada de Persistência).....5
2.4. Model (Camada de Domínio).....6
3. Conclusão.....7
4. Referências Bibliográficas.....8

1. Visão Geral da Arquitetura

1.1. Padrão Arquitetural Adotado

A arquitetura escolhida para o backend é a Arquitetura em Camadas (Layered Architecture), um dos padrões mais utilizados no desenvolvimento de sistemas. Nesse modelo, as classes são organizadas em módulos maiores, chamados de camadas, que se relacionam de forma hierárquica.

Esse padrão foi adotado por:

- Separar responsabilidades de forma clara.
- Facilitar a manutenção e evolução do sistema.
- Melhorar a organização e o entendimento do código.

Optou-se por uma arquitetura em quatro camadas, sendo elas:

- Camada de Apresentação (Controller): responsável pela interação com o usuário ou outras aplicações.
- Camada de Negócio (Service): onde se encontram as regras de negócio.
- Camada de Persistência (DAO): responsável pela comunicação com o banco de dados.
- Camada de Domínio (Model): representa os dados do sistema.

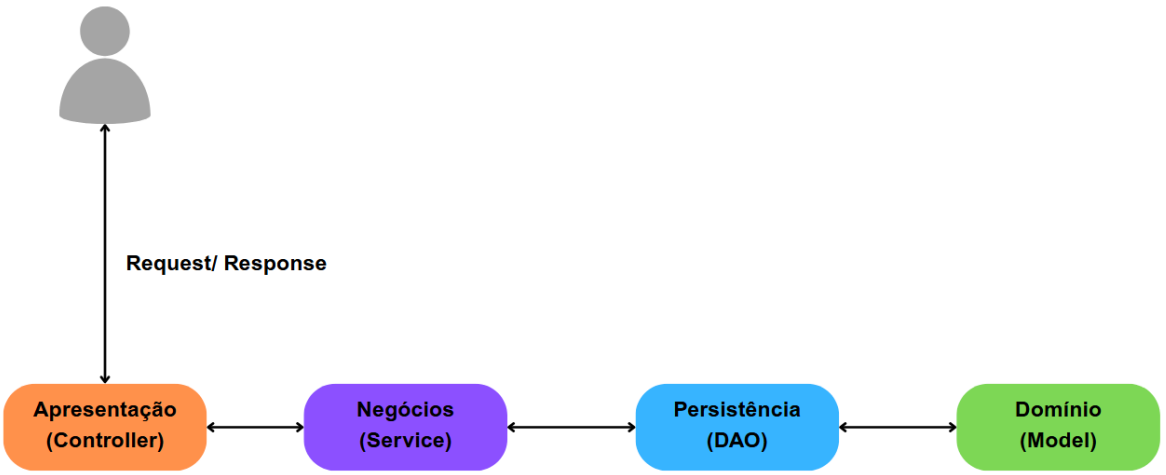


Figura 1 - Exemplo de como as camadas se relacionam entre si

1.2. Organização do Projeto

A estrutura de pastas do projeto segue o padrão de arquitetura em camadas para garantir a separação de responsabilidades.

- controller: Recebe as requisições web (HTTP) do frontend.
- dao: Executa as consultas SQL para interagir com o banco de dados.
- model: Representa as estruturas de dados (ex: User).
- service: Contém a lógica de negócio e as regras da aplicação.

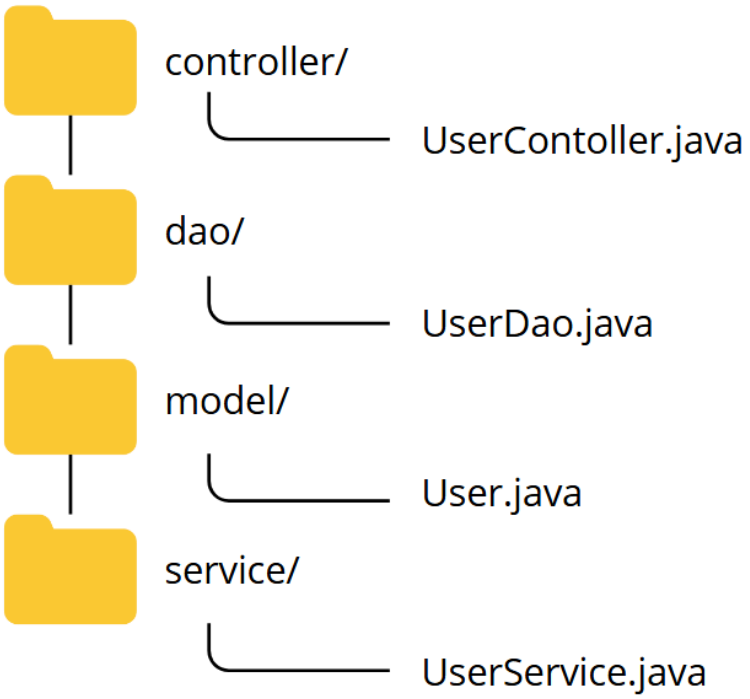


Figura 2 - Organização das pastas do projeto

```
None
com.pingo/
├── controller/
│   └── UserController.java
│
├── dao/
│   └── UserDAO.java
│
├── model/
│   └── User.java
│
└── service/
    └── UserService.java
```

2. Definição das Camadas do Backend

A aplicação backend será dividida nas seguintes camadas, cada uma com uma responsabilidade única.

2.1. Controller (Camada de Apresentação)

Tem como responsabilidade lidar exclusivamente com a comunicação web (HTTP) recebida do frontend React, extraindo os dados e chamando a camada de Service correspondente. É responsável por formatar a resposta HTTP (JSON) de volta para o cliente.

Quando o usuário faz uma requisição chegará no UserController na camada de Apresentação.

Exemplo de implementação:

```java
@RestController
@RequestMapping("/api")
public class UserController {

    private final UserService userService;

    @GetMapping("/users/{userId}")
    public User getUserById(@PathVariable("userId") Long userId) {
        return userService.getUserById(userId);
    }
}
```

2.2. Service (Camada de Negócio)

Esta camada fará o processamento e validação de dados,aplicando as regras de negócios e interagindo com a camada de persistência para armazenar e recuperar dados. Ela é uma ponte entre a Persistência e a Apresentação.

Exemplo de implementação:

```java
@Service
public class UserServiceImpl implements UserService {

    private final UserDAO userDAO;

    public UserServiceImpl(UserDAO userDAO) {
        this.userDAO = userDAO;
```

```
    }

    @Override
    @Transactional(readOnly = true)
    public User getUserById(Long userId) {
        if (userId == null || userId <= 0) {
            throw new InvalidParameterException("ID de usuário inválido.");
        }
        return userDAO.findById(userId)
                .orElseThrow(() -> new UserNotFoundException("Usuário não
encontrado."));
    }
}
```

2.3. Data Access Object (Camada de Persistência)

Essa camada será utilizada para a implementação do acesso a dados de forma manual, conforme o que foi instruído para atender as necessidades dos desenvolvedores juniores.

Aqui estarão as interações com o Banco de Dados, contendo operações CRUD e demais queries necessárias.

Exemplo de implementação:

Java

```
@Repository
public class UserDAOImpl implements UserDAO {

    private final DataSource dataSource;

    public UserDAOImpl(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @Override
    public void save(User user) {
        String sql = "INSERT INTO users (nickname, password) VALUES (?, ?)";

        try (Connection conn = dataSource.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, user.getNickname());
            stmt.setString(2, user.getPassword());
```

```
            stmt.executeUpdate();

        } catch (SQLException e) {
            throw new RuntimeException("Erro ao salvar usuário no banco de
dados.", e);
        }
    }

}
```

2.4. Model (Camada de Domínio)

Nessa camada serão representadas as estruturas de dados do jogo, que futuramente serão mapeadas para as tabelas do banco de dados.

Exemplo de implementação:

```java
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String nickname;

    @Column(nullable = false)
    private String password;

    public User() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getNickname() { return nickname; }
    public void setNickname(String nickname) { this.nickname = nickname; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
```

```
}
```

3. Conclusão

É importante ressaltar que essa é uma versão inicial da arquitetura do sistema, que será aprimorada no futuro, e que os exemplos de código apresentados foram retirados de diversas fontes de referência e tutoriais, servindo a um propósito puramente ilustrativo. Eles não representam a totalidade de um sistema real e funcional, mas sim um guia para a implementação das responsabilidades de cada camada.

4. Referências Bibliográficas

- CALLICODER. spring-boot-react-oauth2-social-login-demo: spring-social/src/main/java/com/example/springsocial/repository/UserRepository.java. [S. l.]: GitHub, 2019. Disponível em: https://github.com/callicoder/spring-boot-react-oauth2-social-login-demo/blob/master/spring-social/src/main/java/com/example/springsocial/repository/UserRepository.java. Acesso em: 1 set. 2025.
- DEUTNER, Gabriel. Arquitetura Backend: A Arquitetura em Camadas. Medium, 2 maio 2022. Disponível em: https://medium.com/@deutnerg/arquitetura-backend-a-arquitetura-em-camadas-01368fc13797. Acesso em: 1 set. 2025.
- ENESGUR, A. Java-Spring-Boot-Reactjs-Boilerplate: BE Java/invoice-service/src/main/java/com/emlakjetcase/invoiceservice/repositories/UserRepository.java. [S. l.]: GitHub, 2022. Disponível em: https://github.com/aenesgur/Java-Spring-Boot-Reactjs-Boilerplate/blob/master/BE%20Java/invoice-service/src/main/java/com/emlakjetcase/invoiceservice/repositories/UserRepository.java. Acesso em: 1 set. 2025.

- GEEKSFORGEEKS. Spring Boot Architecture. GeeksforGeeks, 23 ago. 2023. Disponível em: https://www.geeksforgeeks.org/springboot/spring-boot-architecture. Acesso em: 1 set. 2025.
- JHIPSTER. JHipster Start. [S. l.], 2025. Disponível em: https://start.jhipster.tech. Acesso em: 1 set. 2025.
- MILANI, Andre. Construindo projetos em camadas usando Java. DevMedia, 17 maio 2008. Disponível em: https://www.devmedia.com.br/construindo-projetos-em-camadas-usando-java/6418. Acesso em: 1 set. 2025.
- PATIL, Uday. Spring Boot Architecture. Medium, 23 fev. 2023. Disponível em: https://medium.com/@udaypatil318/spring-boot-architecture-39935654ce5c. Acesso em: 1 set. 2025.
- VALENTE, Marco Tulio. Capítulo 7: Arquitetura em Camadas. In: VALENTE, Marco Tulio. Engenharia de Software Moderna. [S. l.], 2021. Disponível em: https://engsoftmoderna.info/cap7.html. Acesso em: 1 set. 2025.